

Path Traversal

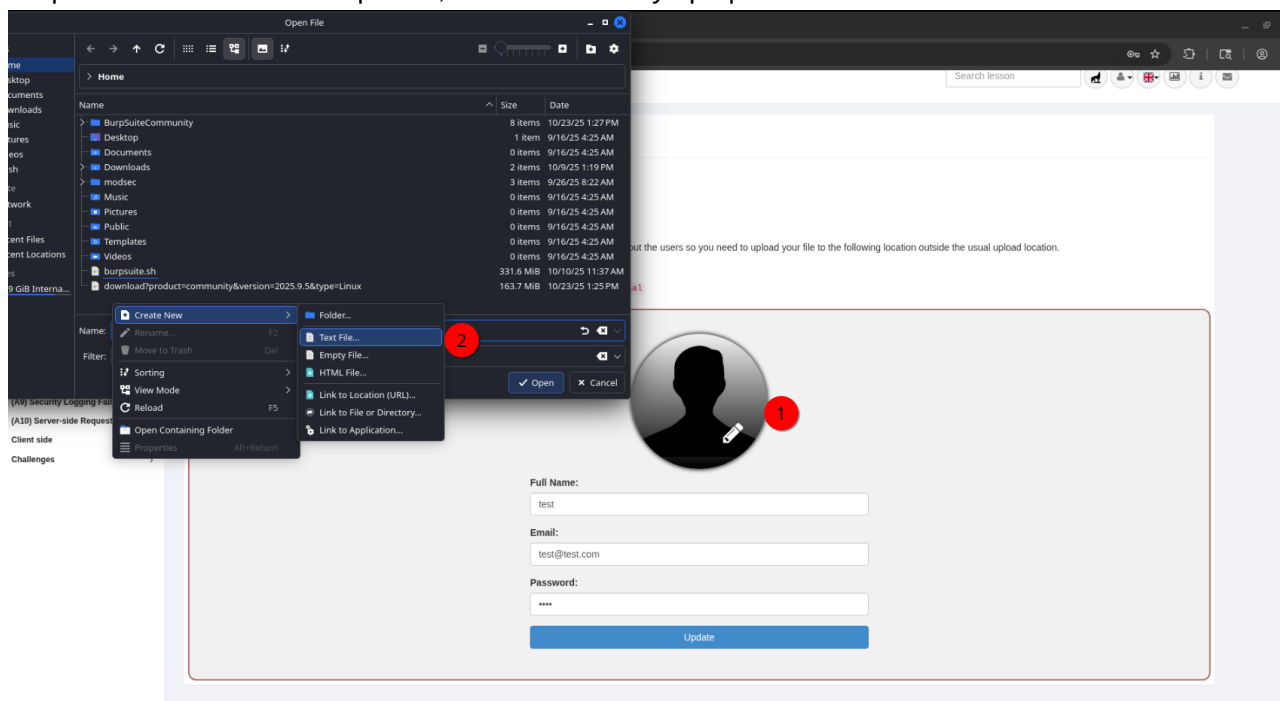
Атака с обходом пути (также известная как обход каталога) направлена на доступ к файлам и каталогам, хранящимся за пределами корневой папки веб-сайта. Манипулируя переменными, ссылающимися на файлы, с помощью последовательностей «точка-точка-слеш (../)» и их вариаций, или используя абсолютные пути к файлам, можно получить доступ к произвольным файлам и каталогам, хранящимся в файловой системе, включая исходный код или конфигурацию приложений, а также критически важные системные файлы. Следует отметить, что доступ к файлам ограничен системным контролем доступа (например, в случае заблокированных или используемых файлов в операционной системе Microsoft Windows).

Эта атака также известна как “dot-dot-slash”, “directory traversal”, “directory climbing” и “backtracking”.

База

Для прохождения задания необходимо развернуть WebGoat, как это было показано в предыдущем уроке. А так же запустить Burpsuite для анализа трафика.

1. Переходим к занятию "(A3) Injection" -> "Path Traversal"
2. Внимательно читаем описание недостатка, которое может привести к уязвимости.
3. Переходим во вторую вкладку и начинаем отправлять данные из формы, которая имитирует обновление картинки пользователя. Перед этим предварительно выбираем файл, который будет отправляться вместе с запросом, нажав на иконку профиля.



4. Посмотрим какие данные передаются при нажатии на кнопку "Update"

The screenshot shows the Burp Suite interface. The 'HTTP history' tab is active, displaying a table of intercepted requests. The selected request (ID 1051) is a POST to `/WebGoat/PathTraversal/profile-upload`. A red arrow points to this request in the table. Below the table, the 'Request' details are shown in 'Pretty' format:

```
1 POST /WebGoat/PathTraversal/profile-upload HTTP/1.1
2 Host: debian:8081
3 Content-Length: 408
4 X-Requested-With: XMLHttpRequest
5 Accept-Language: en-US,en;q=0.9
6 Accept: */*
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryZBVPjBA8t9haVqxy
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
9 Origin: http://debian:8081
10 Referer: http://debian:8081/webGoat/start.mvc?username=useruser
11 Accept-Encoding: gzip, deflate, br
12 Cookie: JSESSIONID=C7170917050308EED899C8B489429A5F
13 Connection: keep-alive
14
15 -----WebKitFormBoundaryZBVPjBA8t9haVqxy
16 Content-Disposition: form-data; name="uploadedFile"; filename="Text File.txt"
17 Content-Type: text/plain
18
19
20 -----WebKitFormBoundaryZBVPjBA8t9haVqxy
21 Content-Disposition: form-data; name="fullName"
22
23 test
24 -----WebKitFormBoundaryZBVPjBA8t9haVqxy
25 Content-Disposition: form-data; name="email"
26
27 test@test.com
28
```

The 'Response' details show an HTTP 200 status with a 'Content-Type: application/json' and a JSON body:

```
1 HTTP/1.1 200
2 Content-Type: application/json
3 Date: Thu, 23 Oct 2025 18:15:41 GMT
4 Keep-Alive: timeout=60
5 Connection: keep-alive
6 Content-Length: 233
7
8 {
9   "lessonCompleted": false,
10  "feedback": "File appears to be empty please upload a non empty file",
11  "feedbackArgs": null,
12  "output": null,
13  "outputArgs": null,
14  "assignment": "ProfileUpload",
15  "attemptWasMade": true
16 }
```

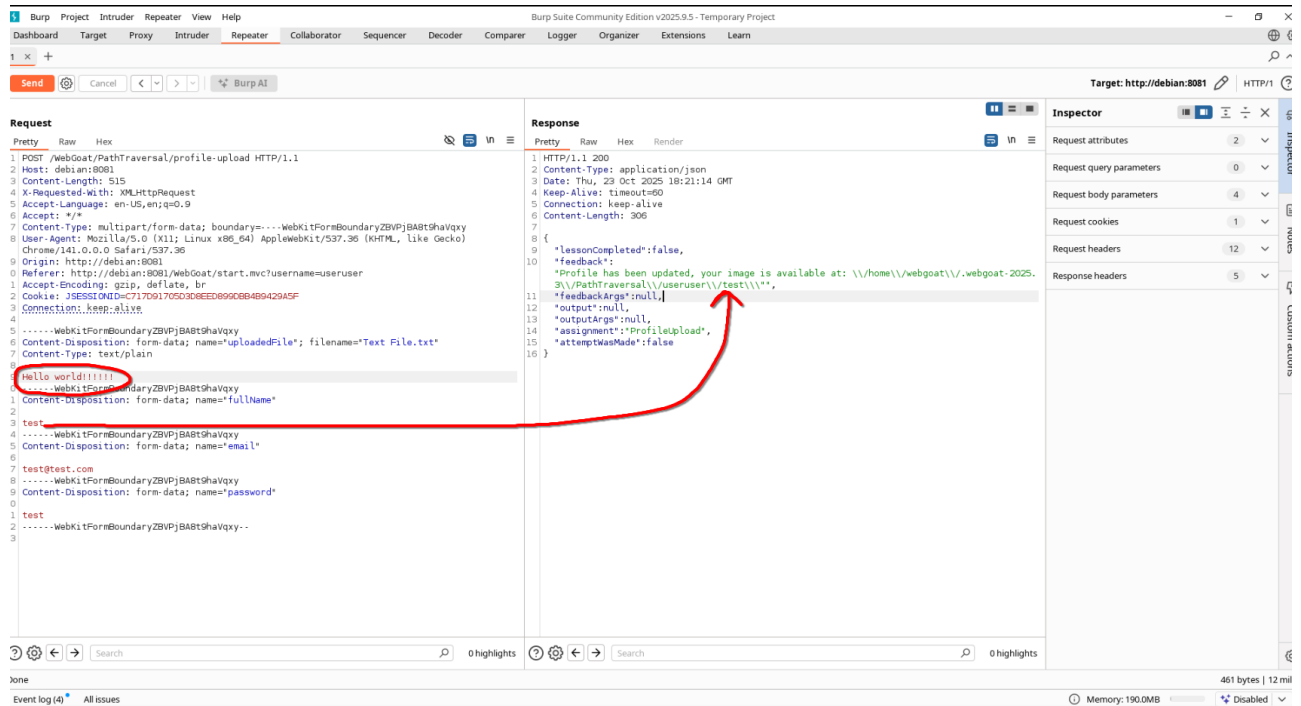
5. При отправке формы передаётся `Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryZBVPjBA8t9haVqxy` Это значит, что информация может передаваться не URL кодированная, с новыми строками и другими символами. При этом все параметры будут разделены специальным маркером `----WebKitFormBoundaryZBVPjBA8t9haVqxy`. Названия передаваемых параметров отражены в поле `name`. При передаче файла также указывается его название в поле `filename`.

6. Нам вернуло ошибку, что мы отправили пустой файл, попробуем это исправить и отправить какие-то данные с помощью Burpsuite. Для этого жмём ПКМ по запросу и выбираем пункт "Send To Repeater".

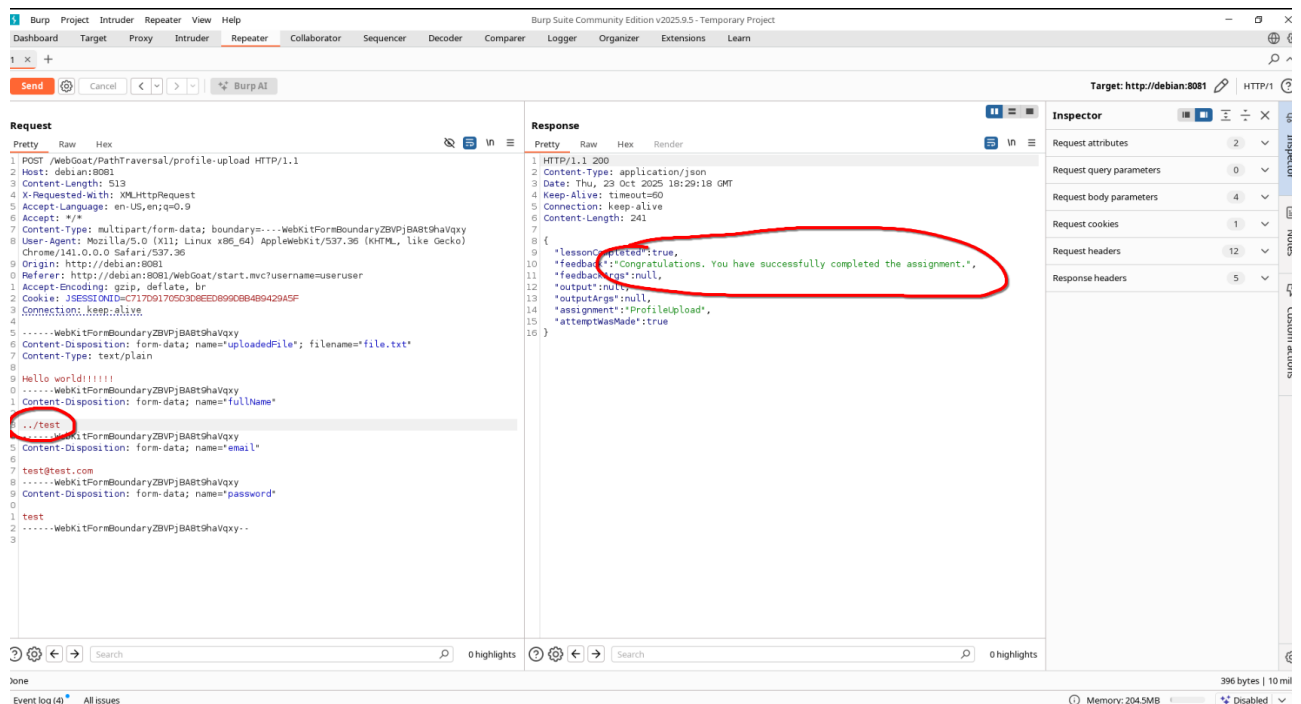
The screenshot shows the Burp Suite interface with the same HTTP history table. A right-click context menu is open over the selected request (ID 1051). The option 'Send to Repeater' is highlighted with a red circle. The menu also includes options like 'Add to scope', 'Scan', 'Send to Intruder', 'Send to Sequencer', 'Send to Organizer', 'Send to Comparer (request)', 'Send to Comparer (response)', 'Show response in browser', 'Request in browser', 'Engagement tools (Pro version only)', 'Show new history window', 'Add notes', 'Highlight', 'Delete item', 'Clear history', 'Copy URL', 'Copy as curl command (bash)', 'Copy links', 'Save item', and 'Proxy history documentation'.

7. После чего переходим во вкладку "Repeater".

8. Во вкладке Repeater добавим немного содержимого в отправляемый запрос и отправим изменённый запрос с помощью кнопки "Send"



9. На изображении выше видно, что в ответе вернулся путь, куда положились отправленная фотография, и в данном пути фигурирует имя нашего пользователя. На данный момент файлы кладутся в следующий файл `/home/webgoat/.webgoat-2025.3/PathTraversal/useruser/test`. А по заданию нам необходимо создать файл в директории `/home/webgoat/.webgoat-2025.3/PathTraversal`. Это можно сделать, если в качестве имени передать `../test`, что приведёт исходный путь `/home/webgoat/.webgoat-2025.3/PathTraversal/useruser/test` к следующему виду `/home/webgoat/.webgoat-2025.3/PathTraversal/useruser/..test`.



Обход защиты

При проверке путей некоторые разработчики могут проверять путь на наличие символов `../` и вырезать их в случае обнаружения, однако данную защиту можно обойти, если отправить, к примеру

..././, тогда в данном случае программа найдёт подстроку ..././, вырежет её и оставит необходимые нам символы ./.. Попробуем данный способ обхода защиты на практике.

1. Отправляем запрос как в прошлый раз и смотрим, что отправляется на сервер.

The screenshot shows the Burp Suite interface with a list of HTTP history items. The selected item is a POST request to `/WebGoat/PathTraversal/profile-upload-fix`. The request body is a multipart/form-data payload. The response is a JSON object with the following structure:

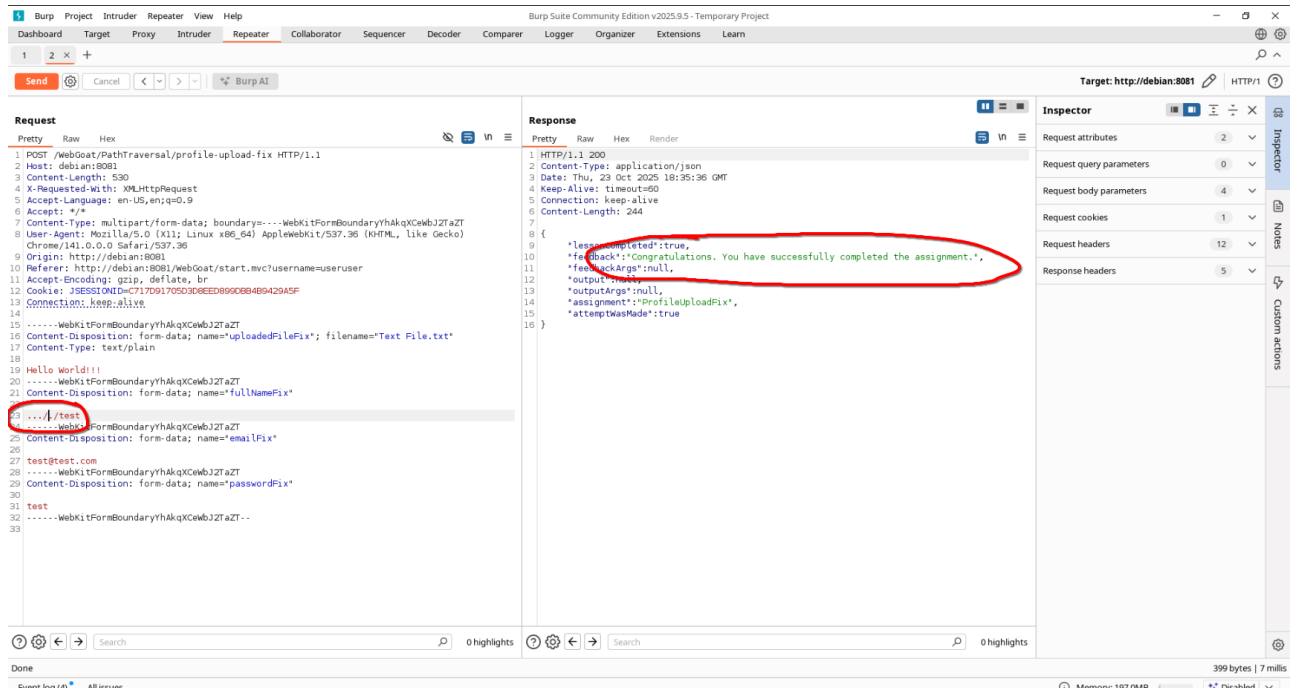
```
{  "lessonCompleted": false,  "feedback": "File appears to be empty please upload a non empty file",  "feedbackArgs": null,  "output": null,  "outputArgs": null,  "assignment": "ProfileUploadFix",  "attemptWasMade": true}
```

2. Поля уже известные и порядок действий тоже. Поэтому попробуем отправить данный запрос в Repeater и изменить поле `fullNameFix` на `../test`

The screenshot shows the Burp Suite Repeater interface. The selected request is a POST request to `/WebGoat/PathTraversal/profile-upload-fix`. The request body is a multipart/form-data payload. The response is a JSON object with the following structure:

```
{  "lessonCompleted": false,  "feedback": "Profile has been updated, your profile is now available at: //home/webgoat/webgoat-2025.3/PathTraversal/user/../../../../test/...",  "feedbackArgs": null,  "output": null,  "outputArgs": null,  "assignment": "ProfileUploadFix",  "attemptWasMade": false}
```

3. Атака не отработала, и наши две точки были вырезаны. Попробуем по предложенному способу отправить ..././.



4. Атака отработала, потому что скрипт отработал по приведённому выше алгоритму. На самом деле, данный способ обхода иногда работает и при других видах инъекций, таких как SQL.

```
selSELECTect -> select.
```

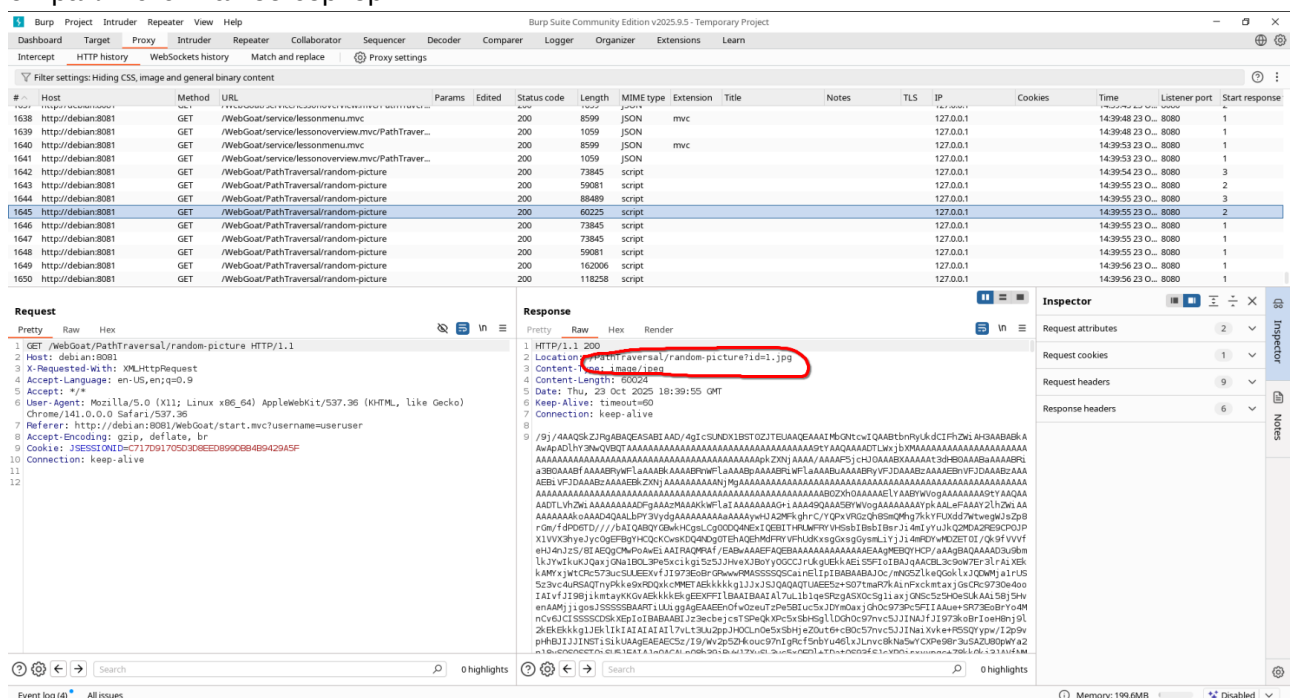
Третий этап

1. Третий этап выполните самостоятельно. Принцип точно такой же, однако немного отличается.

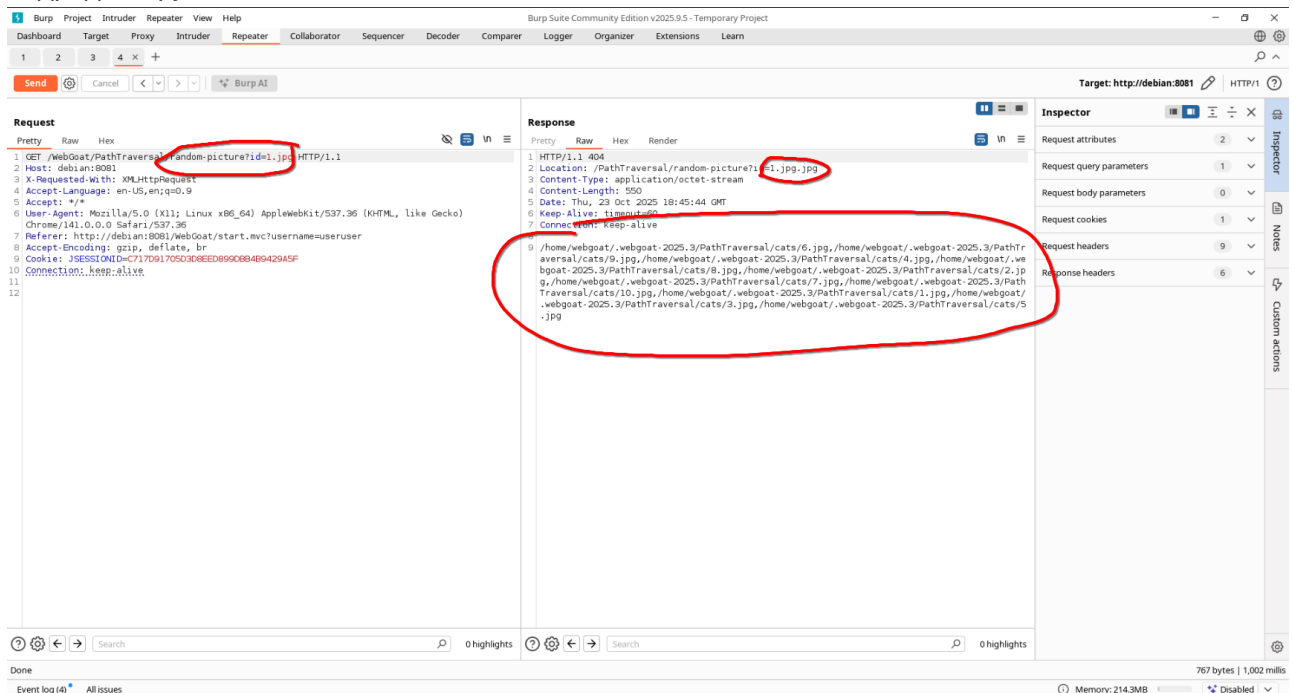
Получение файлов с помощью Path Traversal

Главной уязвимостью при Path Traversal является возможность доступа к другим файлам, к которым доступа быть не должно. Попробуем на примере проэксплуатировать данный недостаток.

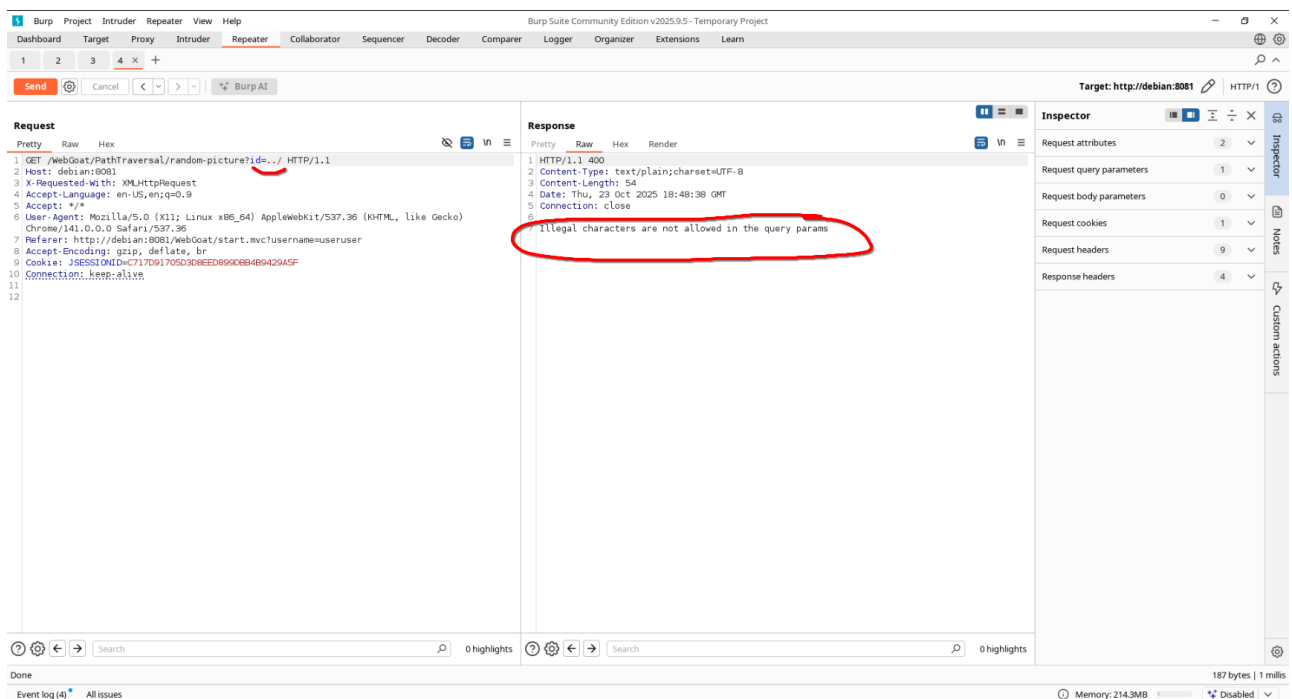
1. Несколько раз нажмём на кнопку "Show random cat picture" и посмотрим какие запросы отправляются на веб-сервер



2. Видим, что с помощью заголовка **Location** отправляется путь, откуда был взят файл, попробуем перейти по этому пути. Для этого отправляем наш запрос в Repeater и подредактируем его.

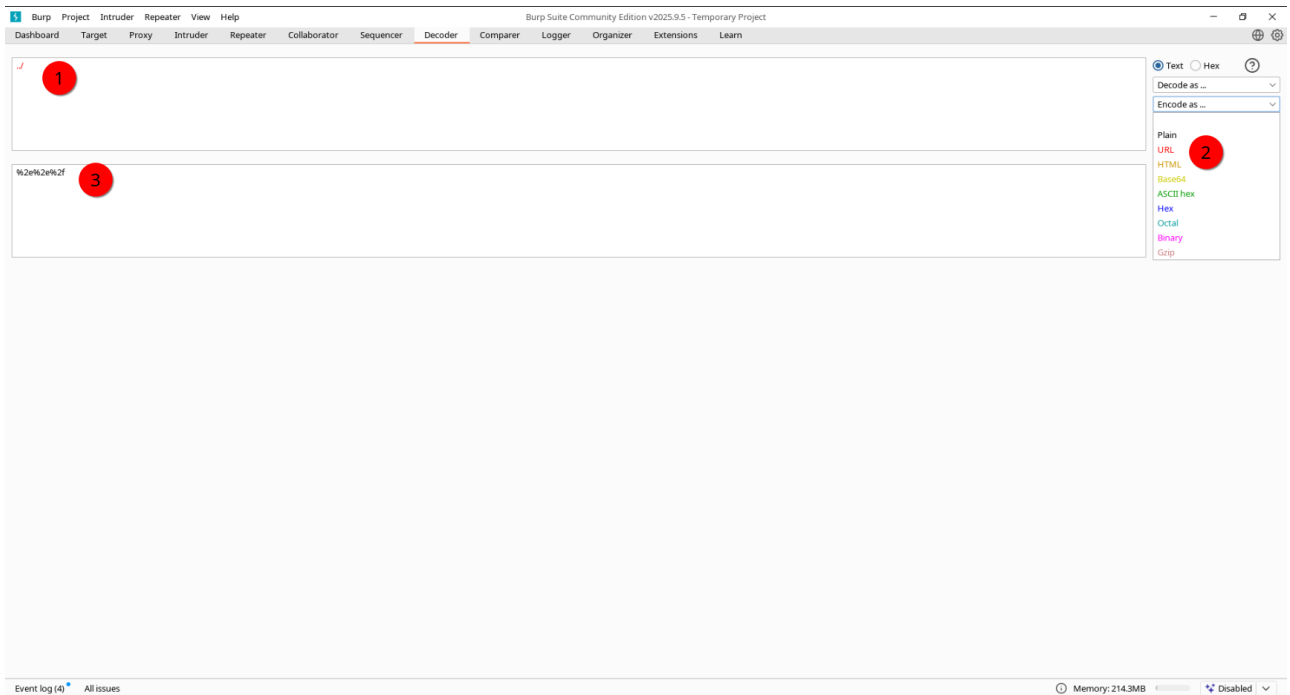


3. Сразу замечаем несколько деталей. Наш путь автоматически дополнился расширением **.jpg**. В случае ошибки, когда сервер не может найти нужный файл он возвращает список файлов в директории. По заданию нам необходимо найти файл **path-traversal-secret.jpg**, однако в данной папке его нет. Воспользуемся недостатком и попробуем выйти из директории, отправив **../**.

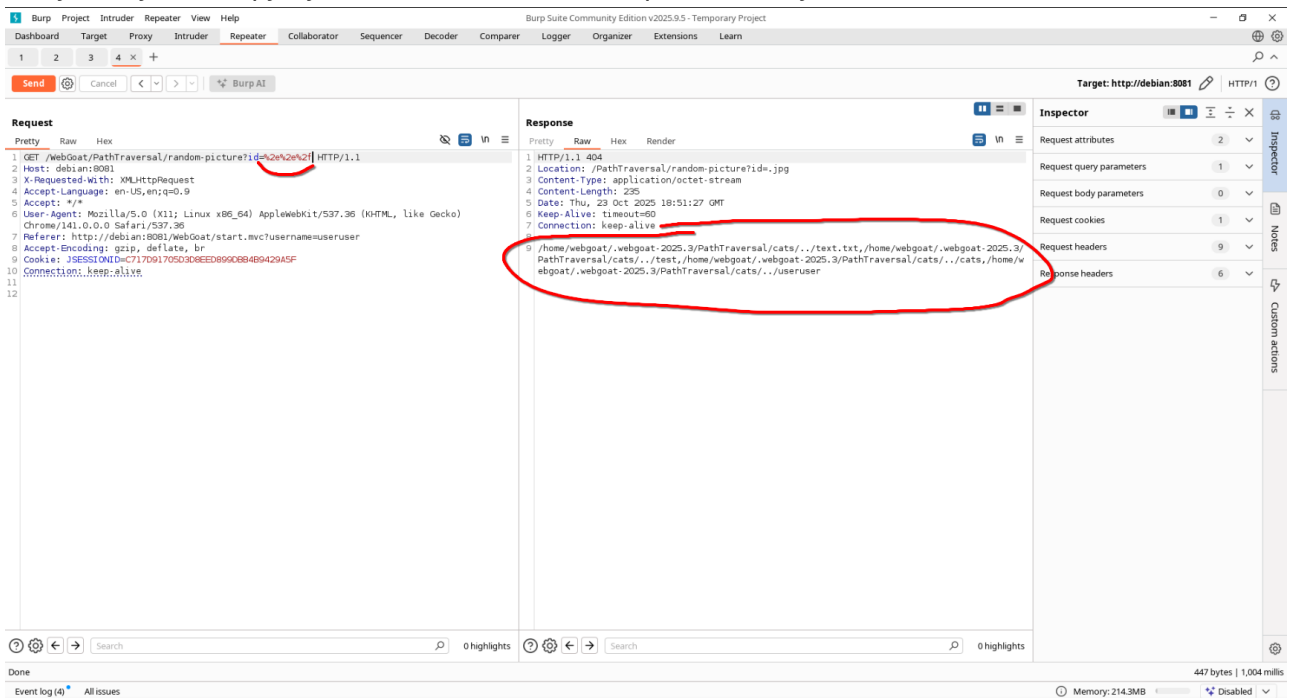


4. Отправить запрос не получилось, по той причине, что параметры, которые передаются через URL должны быть закодированы с помощью метода URL Encoding. Чтобы это сделать можно воспользоваться большим количеством инструментов, один из них встроен прямо в BurpSuite И находится во вкладке "Decoder". Для кодирования выполняем следующий порядок действий: вводим полезную нагрузку в поле(1) выбираем тип кодирования URL(2), копируем нагрузку из

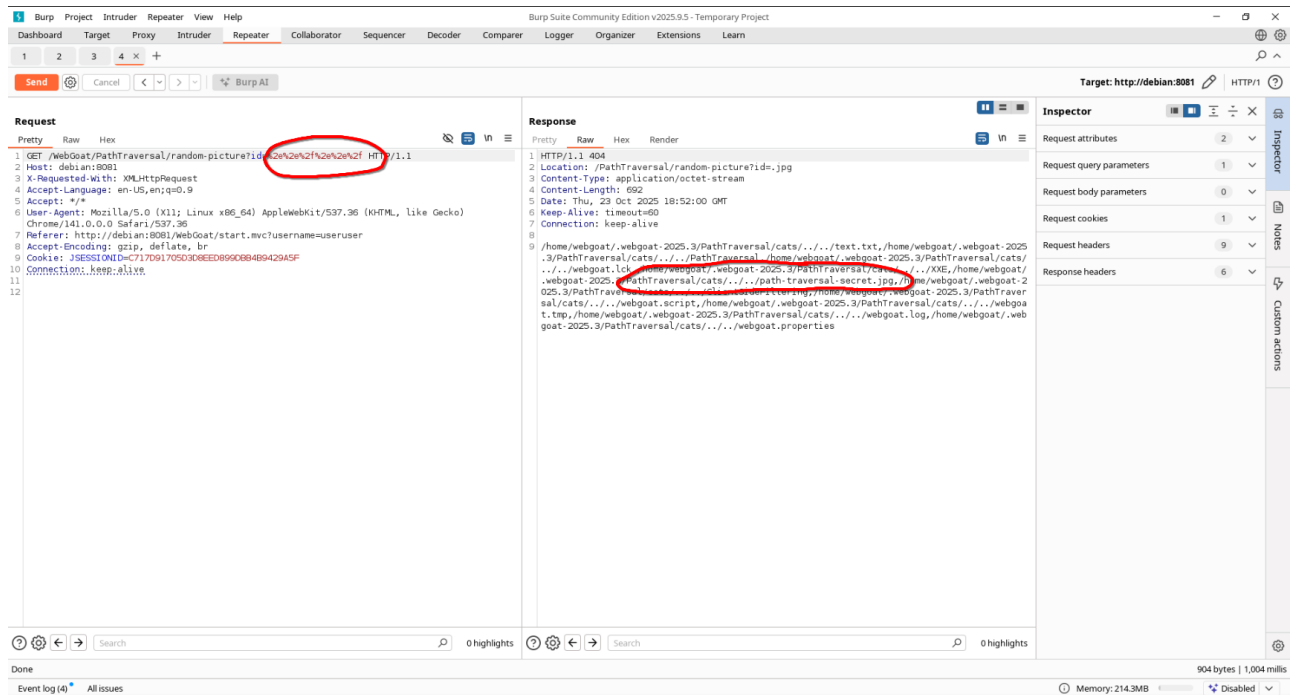
поля снизу(3)



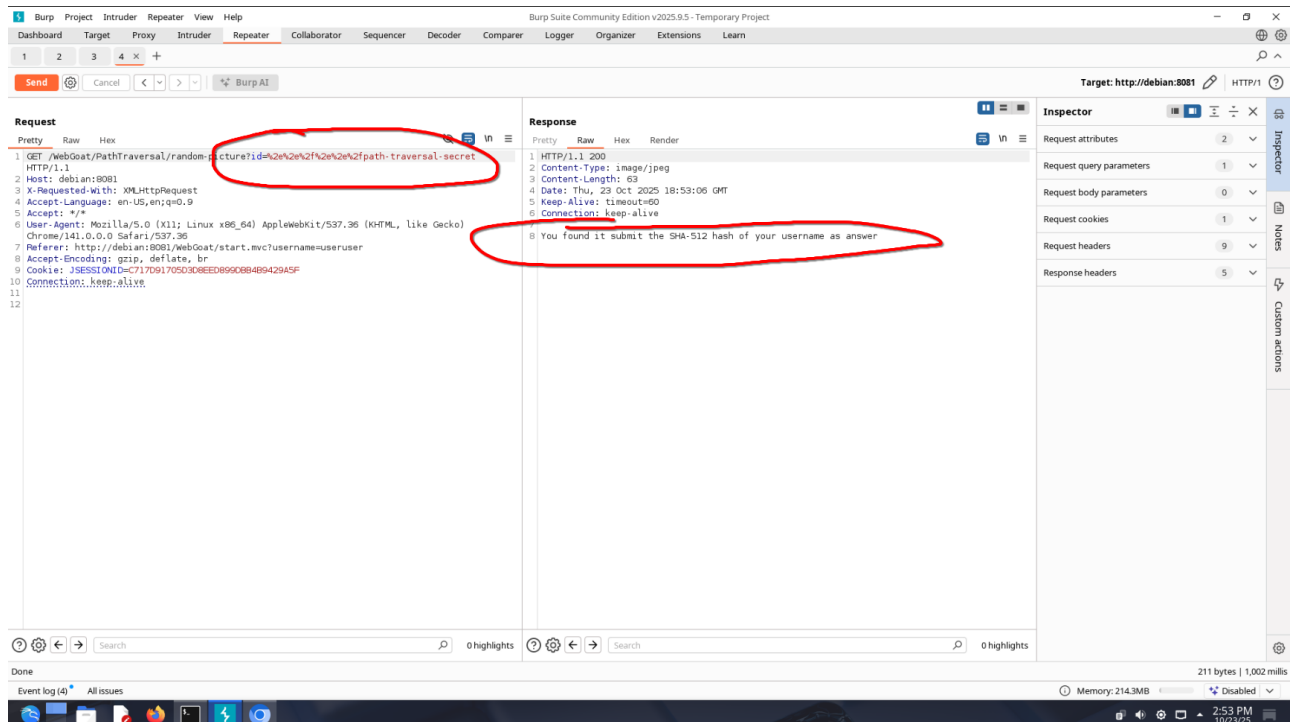
5. Получившуюся нагрузку вставляем в URL и смотрим что получилось.



6. Нужного файла снова нет, поэтому попробуем поискать ещё на уровень повыше `../..`



7. Видим, что есть необходимый нам файл и пытаемся получить к нему доступ (держим в голове, что `.jpg` добавляется автоматически) `../..path-traversal-secret`



8. В качестве ответа к заданию необходимо рассчитать SHA512 от имени пользователя командой `echo -n "useruser" | sha512sum`

Последнее задание самостоятельно!!!!!!